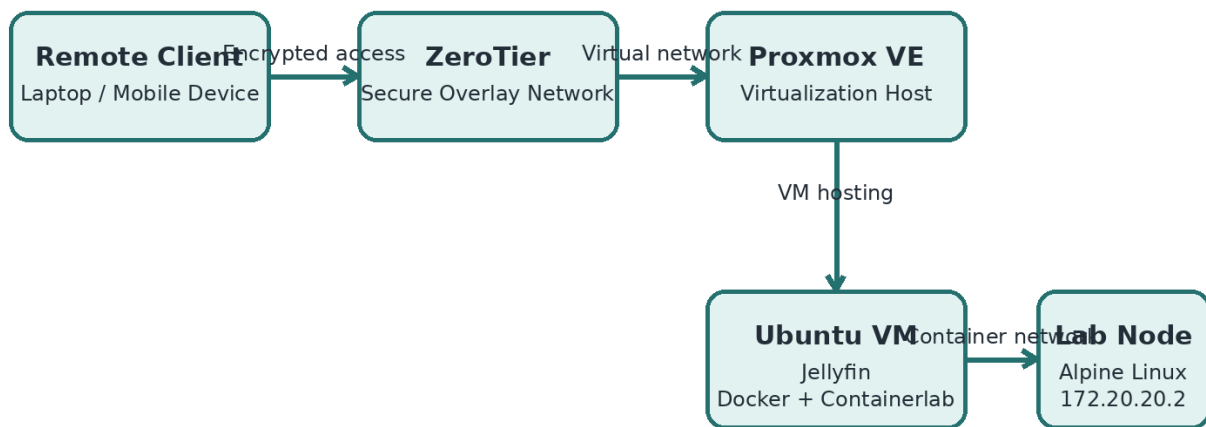


JELLYFIN MEDIA SERVER DEPLOYMENT

Using Proxmox VE, ZeroTier, Docker, and Containerlab

Project System Architecture



Project overview architecture

Final Course Project Report
Prepared by: Maya and Huda
Instructor: Dr. Jehad hamammreh
Submission Date: August 2026

Abstract

This report documents the design, installation, integration, and verification of a virtualized media-server environment built around Jellyfin. The server was deployed on Ubuntu Linux as a virtual machine hosted by Proxmox Virtual Environment (VE). ZeroTier was added to provide remote access through a private virtual network, avoiding dependence on direct public exposure of the Jellyfin service. Docker and Containerlab were then installed on the same Ubuntu server to create a lightweight container-based network laboratory. A simple Alpine Linux node was deployed from a YAML topology file and tested through ICMP connectivity.

The practical evidence collected during the project shows that the Jellyfin service ran successfully, the ZeroTier client joined the project network and received a managed address, Containerlab version 0.77.0 was installed, the topology deployed with the node in a running state, and the connectivity test completed with zero packet loss. The final result is a compact learning environment that combines virtualization, Linux administration, media services, overlay networking, containers, and basic network verification.

Keywords

Proxmox VE, Ubuntu Server, Jellyfin, ZeroTier, Docker, Containerlab, virtualization, remote access, container networking.

Table of Contents

Abstract	2
1. Introduction	4
2. Project Scope and Objectives	5
3. Technical Background	6
4. Requirements and Design	8
5. System Implementation	10
6. Testing and Results	15
7. Challenges and Troubleshooting	18
8. Security, Limitations, and Future Work	20
9. Conclusion	22
References	23
Appendix A - Containerlab Topology	24

List of Figures

- Figure 1. Project system architecture
- Figure 2. Proxmox and Jellyfin server setup
- Figure 3. Jellyfin client application
- Figure 4. ZeroTier network creation
- Figure 5. ZeroTier connection status
- Figure 6. Containerlab version verification
- Figure 7. Containerlab topology deployment
- Figure 8. Container-to-server connectivity test
- Figure 9. Remote access to Jellyfin

1. Introduction

1.1 Project Context

Self-hosted services give users direct control over applications and data while providing practical experience in server administration. A media server is a useful example because it combines operating-system configuration, network access, storage organization, service management, and client connectivity. Jellyfin was selected as the application layer because it provides a web-based and application-based interface for accessing a personal media library.

Rather than installing the service directly on a physical operating system, the project used Proxmox VE as the virtualization layer. This approach separates the media service from the host platform and allows the Ubuntu server to be managed as an independent virtual machine. ZeroTier was used as the remote-access layer, and Containerlab was included to satisfy the networking-laboratory part of the project and to demonstrate container deployment and connectivity testing.

1.2 Problem Statement

A locally hosted Jellyfin server is normally reachable only from the same local network unless additional routing, firewall, port-forwarding, or public-hosting configuration is introduced. Directly exposing a service to the Internet can add configuration complexity and security risk. The project therefore required a method to make the Jellyfin server reachable from authorized devices while retaining a private network model. It also required a reproducible container-based lab that could be deployed and verified from a topology file.

1.3 Project Contribution

The project combines six technologies into one working environment: Proxmox VE, Ubuntu Server, Jellyfin, ZeroTier, Docker, and Containerlab. Its contribution is not a new software product; instead, it is a practical systems-integration implementation that demonstrates how these tools can be installed, connected, tested, documented, and delivered through a GitHub repository.

2. Project Scope and Objectives

2.1 Scope

The project scope includes the creation of an Ubuntu virtual machine, deployment of Jellyfin, creation and authorization of a ZeroTier network membership, installation of Docker and Containerlab, creation of a one-node Alpine Linux topology, and verification of network communication. The scope does not include production-grade storage redundancy, public DNS, a reverse proxy, TLS certificate automation, or a multi-node routing topology.

2.2 Objectives

- Host an Ubuntu Server virtual machine on Proxmox VE.
- Install Jellyfin and verify that the media service operates correctly.
- Create a ZeroTier network and connect the server to it.
- Confirm that ZeroTier assigns the server a managed virtual IP address.
- Install Docker as the container runtime required by Containerlab.

- Install Containerlab and verify its version.
- Create and deploy a valid demo.clab.yml topology.
- Confirm that the deployed node is running and has an assigned IP address.
- Perform an ICMP connectivity test and document the result.
- Organize project evidence, videos, code, presentation, and report in GitHub.

2.3 Success Criteria

Area	Success criterion
Virtualization	Ubuntu VM is available and manageable in Proxmox VE.
Jellyfin	Service starts and can be opened from a client.
ZeroTier	Client reports ONLINE and the network reports OK/PRIVATE.
Containerlab	Version command works and topology deploys.
Connectivity	Ping test receives replies with no packet loss.
Documentation	Screenshots, videos, topology, presentation, and report are available in the repository.

3. Technical Background

3.1 Proxmox VE

Proxmox VE acts as the virtualization platform. It manages virtual machines and provides a web interface for starting, stopping, monitoring, and taking snapshots of guest systems. In this project, it hosts the Ubuntu server that contains all application and lab components.

3.2 Ubuntu Server

Ubuntu Server provides the Linux environment used to install Jellyfin, ZeroTier, Docker, and Containerlab. It also provides systemd service management, package management, networking commands, and the shell environment used throughout the implementation.

3.3 Jellyfin

Jellyfin is the main application service. It provides a media-server backend and user interfaces that allow clients to access configured content. The project evidence includes both server setup and client-side operation.

3.4 ZeroTier

ZeroTier creates a virtual Layer-2/Layer-3 network between authorized devices. Each member joins using a network identifier and must be approved by the network controller. After authorization, a managed address is assigned and the device can communicate with other members of the virtual network.

3.5 Docker

Docker is the container runtime used by Containerlab. Containerlab translates the YAML topology into Docker networks and containers, allowing a repeatable lab to be created and removed from a declarative file.

3.6 Containerlab

Containerlab is a network-lab orchestration tool. The topology file specifies nodes, container kinds, and images. In this project, a single Alpine Linux node named linux1 was deployed to demonstrate the lab workflow and test connectivity.

3.7 Technology Roles

Layer	Technology	Role
Physical/Host	Laptop and Proxmox VE	Runs and manages the virtualized environment.
Guest OS	Ubuntu Server	Hosts all project services.
Application	Jellyfin	Provides the media service.
Remote Network	ZeroTier	Provides private remote connectivity.
Container Runtime	Docker	Runs the lab containers.
Lab Orchestration	Containerlab	Creates the topology from YAML.

4. Requirements and System Design

4.1 Functional Requirements

- The system shall host Jellyfin in an Ubuntu virtual machine.
- The server shall be reachable from an authorized remote device through ZeroTier.
- The system shall support Docker containers.
- The system shall deploy a Containerlab topology from a YAML file.
- The deployed node shall be testable using standard network commands.

4.2 Non-Functional Requirements

- The implementation should use free or open-source tools.
- The configuration should be reproducible and documented.
- The remote-access mechanism should not require exposing Jellyfin directly through a public IP address.
- The project files should be small enough to store in GitHub, excluding virtual disks and full VM images.

4.3 Architecture

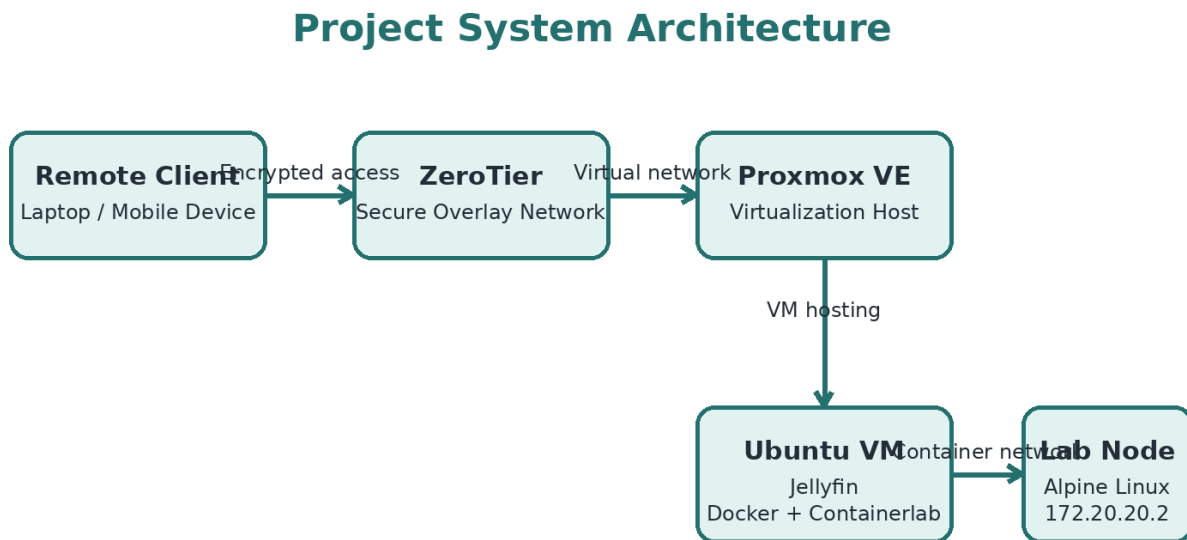


Figure 1. Project system architecture.

The remote client first joins the ZeroTier network. ZeroTier provides the logical connection to the Ubuntu server running inside Proxmox VE. Jellyfin listens on the Ubuntu VM, while Docker and Containerlab create an additional internal container network. The demonstrated Containerlab node received the IPv4 address 172.20.20.2.

4.4 Addressing Observed During Testing

Interface or component	Observed address	Purpose
Ubuntu LAN interface	192.168.68.104 or 192.168.68.114 during different sessions	Local network access to the VM.
ZeroTier interface	10.186.189.47/24	Private overlay-network access.
Containerlab node	172.20.20.2	Container-lab network address.
Containerlab network	172.20.20.0/24	Docker network automatically created by Containerlab.

The local address changed between sessions because the Ubuntu interface was using dynamic addressing. The ZeroTier and Containerlab addresses shown above are taken directly from the captured project evidence.

5. System Implementation

5.1 Proxmox VE and Ubuntu Virtual Machine

The project started with Proxmox VE as the host platform. An Ubuntu virtual machine named Jellyfin-Server was created and accessed through the Proxmox web console. The VM was assigned network access and sufficient storage to install the operating system and application packages. A Proxmox snapshot was also used during the work to preserve a known state before further changes.

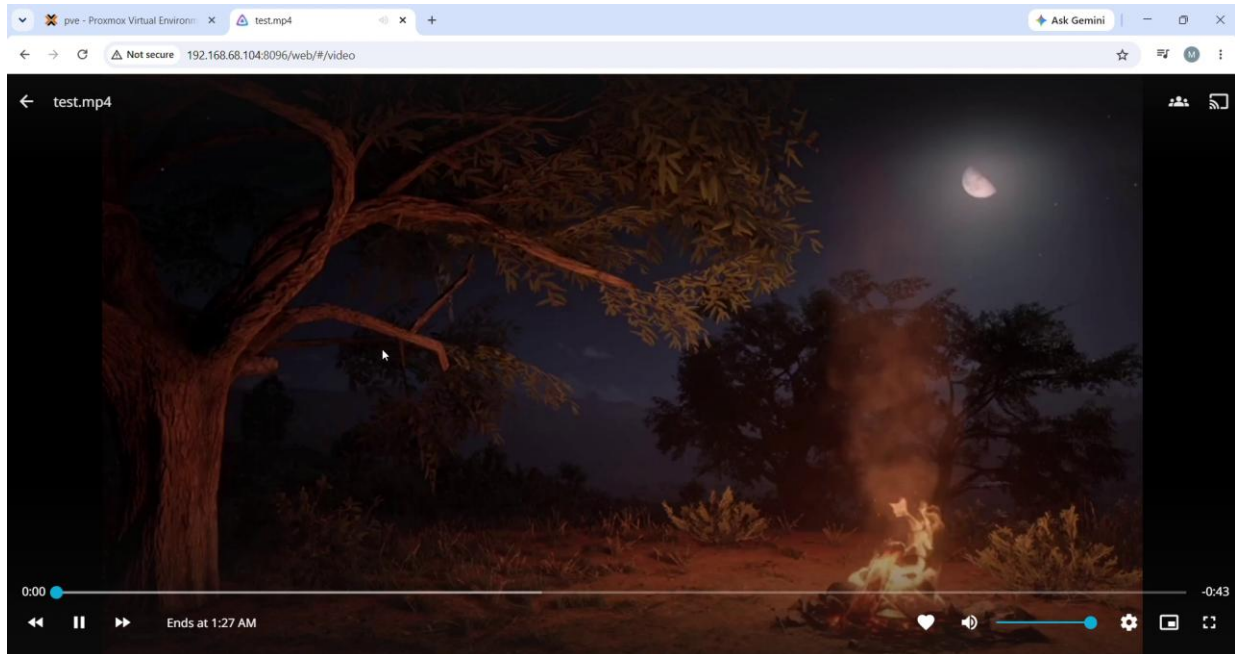


Figure 2. Proxmox console and Jellyfin server setup evidence.

5.2 Jellyfin Installation and Service Verification

Jellyfin was installed inside the Ubuntu VM and configured as a system service. Service verification showed an active running state and successful startup messages. The client application was then used to confirm that the server could be reached and that the Jellyfin interface loaded correctly.

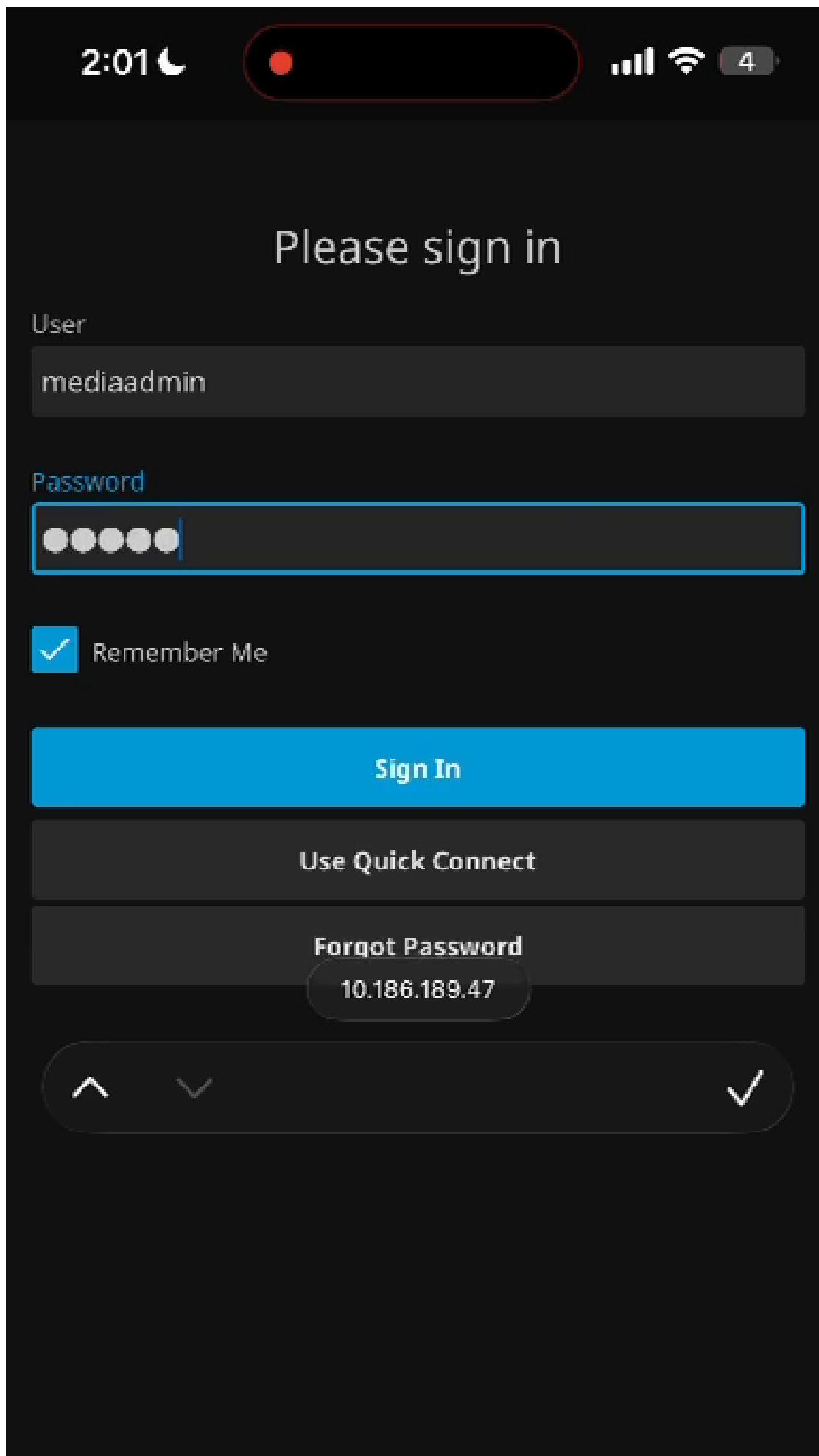


Figure 3. Jellyfin client application operating against the configured server.

5.3 ZeroTier Network Creation

A ZeroTier organization and network were created through the ZeroTier Central web portal. The project network was named Jellyfin. The portal generated the network ID used by client devices to join the same private virtual network. The network identifier appears in the screenshots and should be treated as project configuration rather than a login credential.

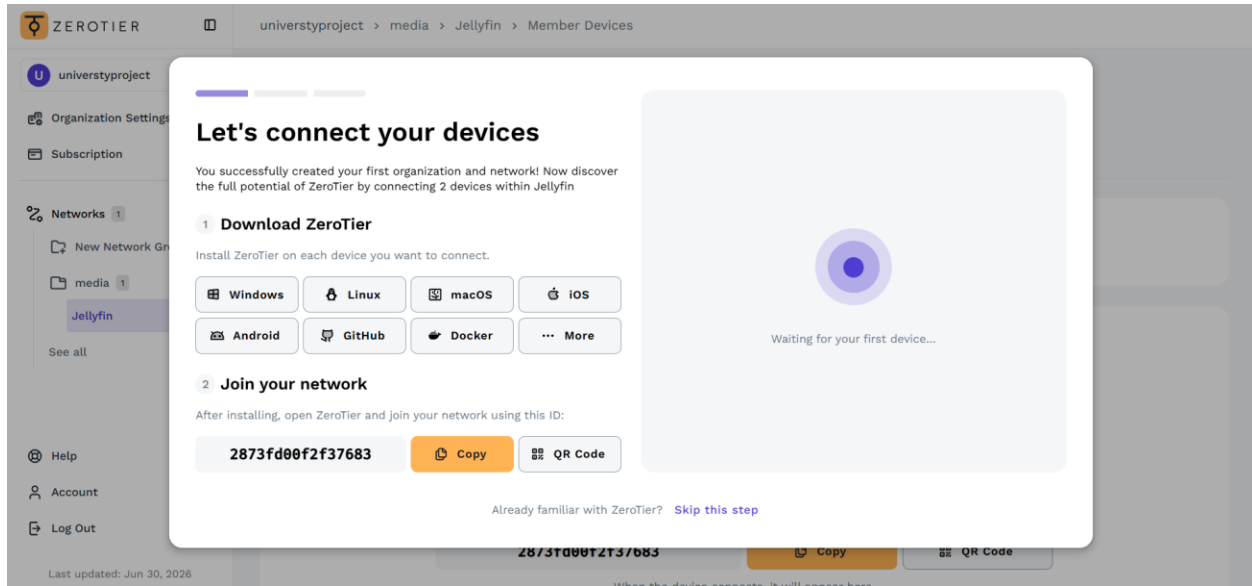


Figure 4. ZeroTier project network and device-connection screen.

5.4 Installing and Joining ZeroTier

The ZeroTier client was installed on the Ubuntu VM and its service was enabled. The status and info commands reported that the client was ONLINE. The server then joined the project network using the network ID. Before authorization, the network status showed ACCESS_DENIED. After the device was authorized from ZeroTier Central, the status changed to OK PRIVATE and the interface received the address 10.186.189.47/24.

```
sudo zerotier-cli status
sudo zerotier-cli info
sudo zerotier-cli join <network-id>
sudo zerotier-cli listnetworks
ip a | grep zt
```

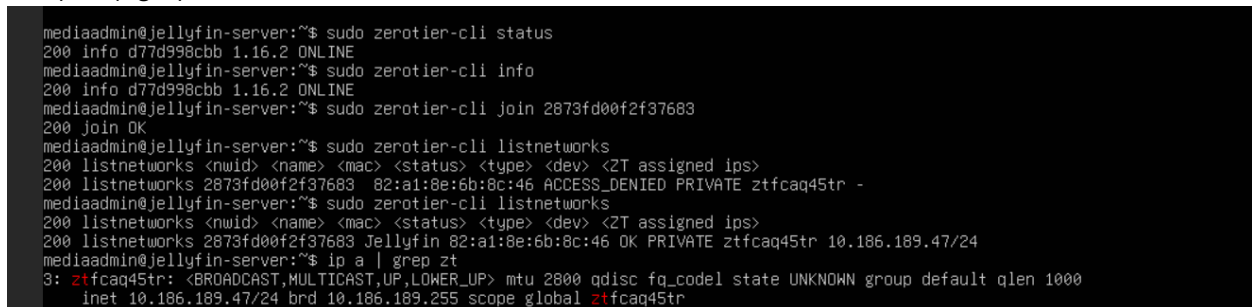


Figure 5. ZeroTier client online, authorized, and assigned a virtual IP address.


```

mediaadmin@jellyfin-server:~/clab-demo$ cat demo.clab.yml
name: demo

topology:
  nodes:
    linux1:
      kind: linux
      image: alpine:latest
mediaadmin@jellyfin-server:~/clab-demo$ sudo containerlab deploy
[sudo: authenticate] Password:
15:26:43 INFO Containerlab started version=0.77.0
15:26:43 INFO Parsing & checking topology file=demo.clab.yml
15:26:43 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24 IPv6 subnet=3fff:172:20:20::/64 MTU=0
15:26:44 INFO Pulling images
V Pulled image docker.io/library/alpine:latest
15:26:51 INFO Creating lab directory path=/home/mediaadmin/clab-demo/clab-demo
15:26:51 INFO Creating container name=linux1
15:26:54 INFO Adding host entries path=/etc/hosts
15:26:54 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-demo.conf

```

Name	Kind/Image	State	IPv4/6 Address
clab-demo-linux1	linux alpine:latest	running	172.20.20.2 3fff:172:20:20::2

```

mediaadmin@jellyfin-server:~/clab-demo$ docker ps
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
mediaadmin@jellyfin-server:~/clab-demo$ sudo containerlab inspect
15:28:08 INFO Parsing & checking topology file=demo.clab.yml

```

Name	Kind/Image	State	IPv4/6 Address
clab-demo-linux1	linux alpine:latest	running	172.20.20.2 3fff:172:20:20::2

Figure 7. Successful Containerlab deployment and inspection.

5.8 Connectivity Test

A shell was opened inside the deployed container using `docker exec`. The container then sent three ICMP echo requests to the Ubuntu server local address. All three packets were received, producing 0% packet loss. This test confirms that the container was running and that communication from the Containerlab network to the server network was functional in the tested environment.

```

sudo docker exec -it clab-demo-linux1 sh
ping -c 3 192.168.68.104
mediaadmin@jellyfin-server:~/clab-demo$ sudo docker exec -it clab-demo-linux1 sh
/ # ping -c 3 192.168.68.104
PING 192.168.68.104 (192.168.68.104): 56 data bytes
64 bytes from 192.168.68.104: seq=0 ttl=63 time=154.524 ms
64 bytes from 192.168.68.104: seq=1 ttl=63 time=60.453 ms
64 bytes from 192.168.68.104: seq=2 ttl=63 time=81.628 ms

--- 192.168.68.104 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 60.453/98.868/154.524 ms

```

Figure 8. Successful container-to-server ping with 0% packet loss.

5.9 Remote Jellyfin Access

A separate client device was connected to the same ZeroTier network and used to access the Jellyfin service. The recorded evidence demonstrates that the Jellyfin interface was reached from the remote device. This verified the main integration goal: making the locally hosted media server available to an authorized external client through the private overlay network.

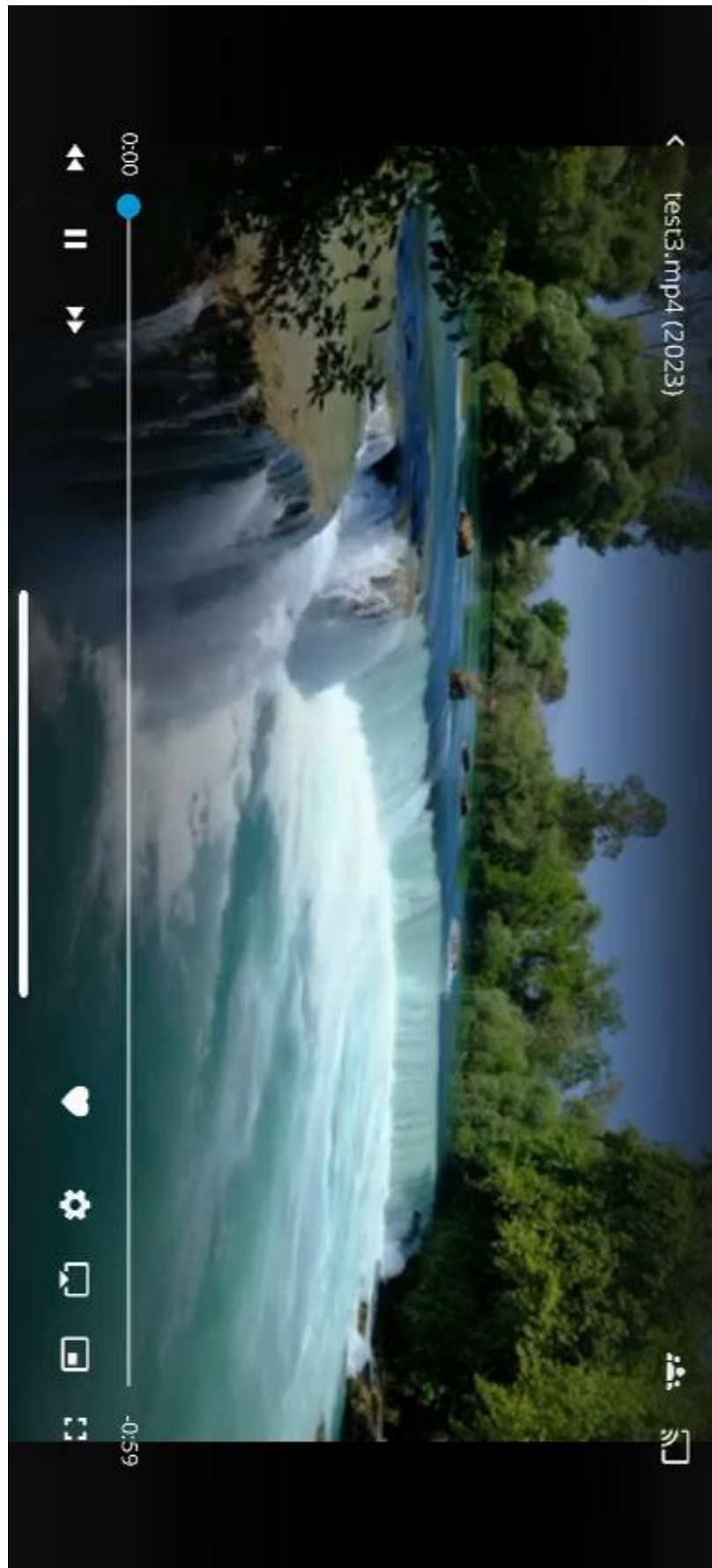


Figure 9. Remote Jellyfin access from an authorized client device.

6. Testing and Results

6.1 Test Plan

Test ID	Test	Method	Expected result
T1	Ubuntu VM availability	Open Proxmox console and log in	VM starts and accepts login.
T2	Jellyfin service	Check systemd status and open client	Service is active and interface loads.
T3	ZeroTier client	Run status and info commands	Client reports ONLINE.
T4	ZeroTier membership	Run listnetworks after authorization	Network reports OK PRIVATE with assigned IP.
T5	Containerlab installation	Run containerlab version	Version information is displayed.
T6	Topology deployment	Run deploy and inspect	Node reports running with an IP address.
T7	Network communication	Ping server from container	Replies received without packet loss.
T8	Remote application access	Open Jellyfin from remote client	Jellyfin interface is reachable.

6.2 Observed Results

Test ID	Observed evidence	Status
T1	Ubuntu login and Proxmox console available.	PASS
T2	Jellyfin service showed active/running and client interface operated.	PASS
T3	ZeroTier status output reported ONLINE.	PASS
T4	Network changed from ACCESS_DENIED to OK PRIVATE and received 10.186.189.47/24.	PASS
T5	Containerlab version 0.77.0 displayed.	PASS
T6	clab-demo-linux1 reported running at 172.20.20.2.	PASS
T7	Three packets transmitted and received; 0% packet loss.	PASS
T8	Remote access video shows Jellyfin reached from another client.	PASS

6.3 Result Analysis

The strongest result is that all major layers were verified independently and together. Proxmox verified the virtualization layer, Jellyfin verified the application layer, ZeroTier verified the remote-network layer, and Containerlab verified the container-lab layer. The ping result demonstrates that the deployed container was not only running but also had usable connectivity beyond its own namespace.

The project is a functional proof of concept rather than a production benchmark. No throughput, latency, concurrent-user, storage-performance, or transcoding-performance measurements were collected. Therefore, the report makes no claim about production capacity. It confirms correct deployment and basic reachability only.

7. Challenges and Troubleshooting

Challenge	Cause or symptom	Resolution
Package manager lock	dpkg frontend lock held by another process during ZeroTier installation.	Waited for the package operation to complete and reran installation instead of deleting the lock file.
ZeroTier command unavailable	Initial installation did not complete, so zerotier-cli was not found.	Completed package installation, enabled the service, and verified ONLINE status.
ZeroTier ACCESS_DENIED	The server had joined the network but was not authorized.	Authorized the member device in ZeroTier Central; status became OK PRIVATE.
Docker permission denied	The user was not yet operating with Docker socket permission.	Used sudo for Docker and Containerlab verification commands.
No topology file found	Containerlab deploy was run before a valid *.clab.yml file was available.	Created demo.clab.yml, checked its contents, and deployed again.
YAML parsing error	The topology file contained malformed YAML during an earlier attempt.	Corrected indentation and preserved the required topology structure.
Port connection refused	An early test used a port that was not accepting connections.	Verified the active service and focused the integration test on confirmed connectivity and remote client access.

7.1 Lessons Learned

- Package-management locks should be allowed to clear naturally; deleting lock files can damage package state.
- ZeroTier membership has two stages: joining the network and authorization by the controller.
- Docker access is controlled through the Docker socket; sudo or group membership is required.
- Containerlab topology filenames must follow the *.clab.yml or *.clab.yaml pattern.
- YAML formatting must be checked carefully because indentation defines structure.
- A running service should be verified with both service status and a real client connection.

8. Security, Limitations, and Future Work

8.1 Security Considerations

ZeroTier reduced the need to expose Jellyfin directly through router port forwarding. Only authorized network members should be allowed to join. The GitHub repository should not contain passwords, authentication tokens, private keys, or full virtual-machine disk images. Screenshots may contain IP addresses and network identifiers; these are useful for evidence but should still be reviewed before public publication.

- Use strong Jellyfin user passwords.
- Remove unused ZeroTier members.
- Apply Ubuntu and Jellyfin security updates.
- Restrict firewall access to required ports and interfaces.
- Use HTTPS or a reverse proxy if the environment becomes production-facing.
- Avoid committing VM disks, credentials, and secret configuration to GitHub.

8.2 Current Limitations

- The Containerlab topology contains only one node.
- The Ubuntu LAN address was dynamically assigned and changed between sessions.
- No high-availability or backup storage design was implemented.
- No automated TLS certificate or DNS configuration was included.
- No formal performance or security-penetration testing was carried out.
- The lab demonstrates connectivity but is not directly integrated into Jellyfin application traffic.

8.3 Future Work

- Create a multi-node Containerlab topology with routers, switches, and client containers.
- Assign a reserved DHCP lease or static address to the Ubuntu VM.
- Add monitoring with Prometheus and Grafana.
- Add a reverse proxy and HTTPS.
- Automate deployment with shell scripts or Ansible.
- Add scheduled Proxmox backups and Jellyfin configuration backups.
- Measure media-streaming performance over local and ZeroTier connections.

9. Conclusion

The project successfully implemented a complete virtualized media-server environment and documented each major step. Proxmox VE provided the host virtualization platform, Ubuntu Server provided the operating environment, Jellyfin provided the media service, ZeroTier provided authorized remote connectivity, Docker provided the container runtime, and Containerlab provided a reproducible network laboratory.

The evidence confirms that the server and networking components were operational. ZeroTier reported an online and authorized connection, Containerlab deployed the Alpine Linux node in a running state, and the container-to-server ping test completed with 0% packet loss. Remote Jellyfin access was also demonstrated from another client device. The project therefore met its stated objectives and provides a

strong foundation for more advanced work in virtualization, remote access, service hosting, and network automation.

References

1. **Jellyfin Project:** <https://jellyfin.org>
2. **Proxmox VE:** <https://www.proxmox.com>
3. **ZeroTier:** <https://www.zerotier.com>
4. **Containerlab Documentation:** <https://containerlab.dev>
5. **Docker Documentation:** <https://docs.docker.com>
6. **Ubuntu Documentation:** <https://docs.ubuntu.com>

Project Evidence

The screenshots, videos, presentation, README file, and demo.clab.yml topology referenced in this report are included in the project GitHub repository prepared for submission.

Appendix A - Containerlab Topology File

File name: demo.clab.yml

```
name: demo

topology:
  nodes:
    linux1:
      kind: linux
      image: alpine:latest
```

Appendix B - Verification Commands

```
systemctl status jellyfin
sudo zerotier-cli status
sudo zerotier-cli info
sudo zerotier-cli listnetworks
ip a | grep zt
containerlab version
sudo containerlab deploy
sudo containerlab inspect
sudo docker exec -it clab-demo-linux1 sh
ping -c 3 192.168.68.104
```

Appendix C - Repository Deliverables

- README.md - project overview and workflow.
- demo.clab.yml - Containerlab topology.
- Screenshots/ - practical evidence.
- Videos/ - Jellyfin, ZeroTier, and remote-access demonstrations.
- Project Presentation.pdf - presentation slides.
- Jellyfin_Complete_Project_Report.docx - editable report.
- Jellyfin_Complete_Project_Report.pdf - submission-ready report.